/bin/threadtest：exec 之后（pid 20 的两份快照），以及 clone 出两个线程之后的结构

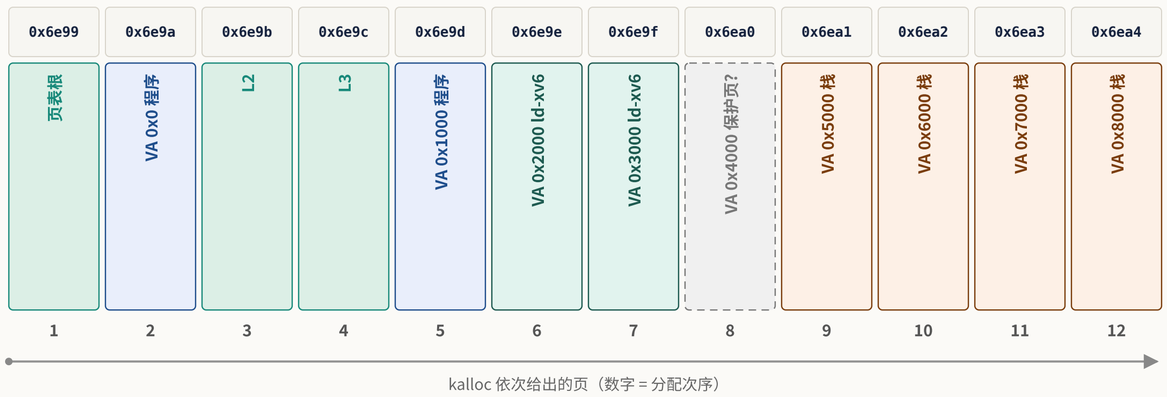
上：快照里的真实数值。下：按 user/threadtest.c、thread.c 与 kernel/proc.c threadclone() 推出的线程结构（线程部分没有快照，地址为示意）

① exec 与第一个系统调用（快照 A / B）

	A：exec-image:first	B：first-syscall:return
struct proc	0xffffff8006eb7000 → pa 0x6eb7000	同左
vmspace	0xffffff8006eb5000 → pa 0x6eb5000	同左
内核栈	槽 15，pa 0x6ec0000	同左
当前内核 sp	+0x940（sys_exec 深处，1728 B）	+0xe80（usertrap+syscall，384 B）
context.sp	+0x760（exec 读盘时睡眠）	+0x690（open 里睡眠）
elr	0x2750 ldso_start	0x2b9c ld-xv6 __syscall+0x20
user sp	0x8f60 argv 表	0x8a30 ldso_start 的栈帧（−0x530）
x0	0x2daa 旧值（argc 尚未写入）	3 open("/lib/libc.so") 的 fd
页表根	0x6e99000	同左（8 个用户页，libc 还没装）

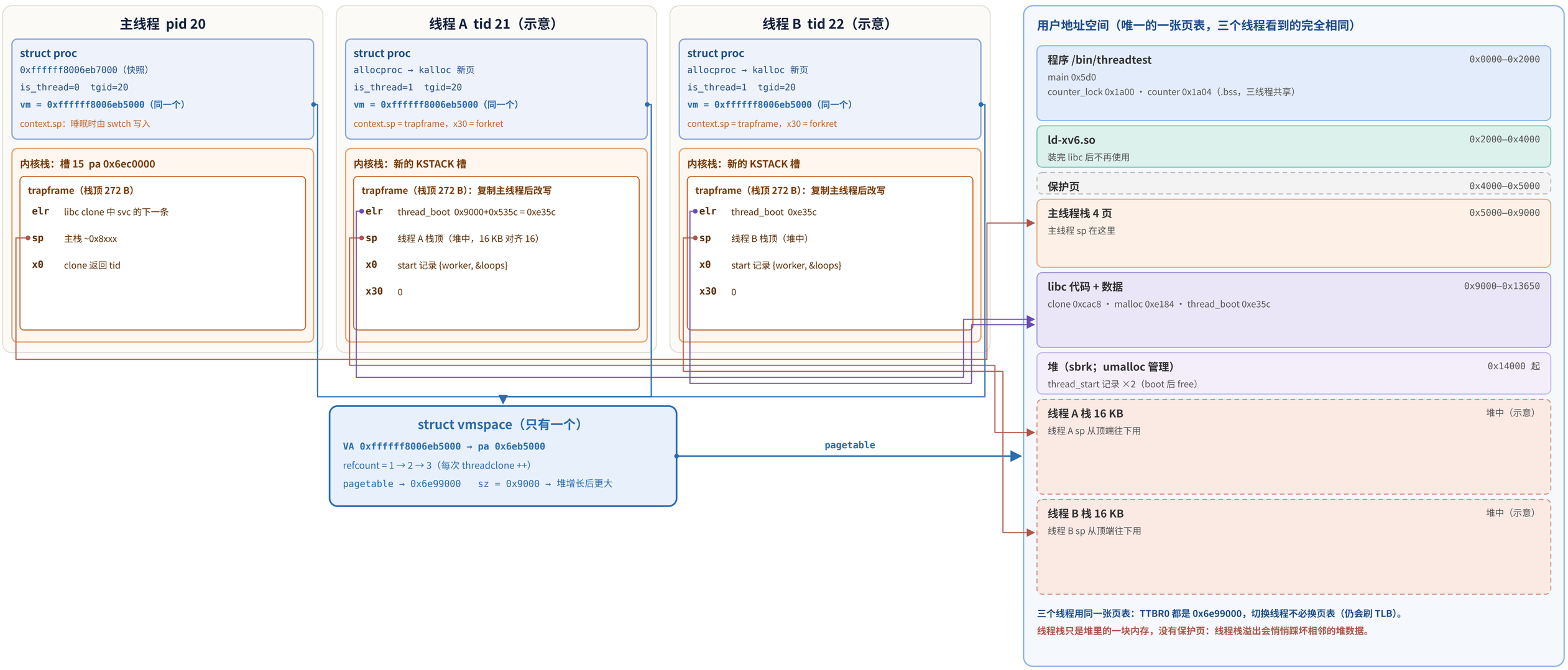
与 ls（pid 17）对比：proc、vmspace、内核栈用的是同一批物理页——ls 退出后被释放，又分给了 pid 20。

② 新页表与用户页的物理分配顺序（这次是递增）



ls 那次是 0x6e8c → 0x6e81 递减；这次 0x6e99 → 0x6ea4 递增。
原因：ls 退出时按顺序把页还回空闲链表，kalloc 后进先出，取回来就倒过来了。
顺序相同：根 → VA0 数据页 → 走表时补的 L2、L3 → 其余用户页。

③ main 调两次 thread_create → clone → threadclone()：三个 proc 共享一个 vmspace



三个线程用同一张页表：TTBR0 都是 0x6e99000，切换线程不必换页表（仍会刷 TLB）。
线程栈只是堆里的一块内存，没有保护页：线程栈溢出会悄悄踩坏相邻的堆数据。

④ 时间线

exec：换上新页表（快照 A），eret 到 ldso_start

ld-xv6：open /lib/libc.so（快照 B，fd 3）→ pread + sbrk 装到 0x9000

main：sync_create(SYNC_MUTEX) 得到 counter_lock

thread_create(A)：malloc 16 KB 栈 + start 记录 → clone(thread_boot, start, 栈顶)

threadclone：allocproc（新 proc、新内核栈、新 trapframe）→ vm->refcount++，np->vm = 同一 vmspace

复制 trapframe，改 elr = thread_boot、sp = 栈顶、x0 = start → make_runnable

thread_create(B)：同上，refcount = 3

线程被调度：swtch → forkret → usertrapret → eret 到 thread_boot → worker

worker：1000 次 sync_wait / counter++ / sync_signal，每 128 次 sched_yield

texit(0)：只拆掉这个线程，refcount−−；main 的 thread_join 等两个线程结束

检查 counter == 2000 → PASS

⑤ 线程之间什么共享、什么各有一份

	共享（同一份）	各有一份
地址空间	vmspace、页表、sz、全部用户页	—
代码与全局变量	程序、libc、counter、counter_lock	—
用户栈	—	主线程：0x5000–0x9000；线程：堆中 16 KB
内核侧	—	struct proc、内核栈、trapframe、context
身份	tgid = 20（getpid 返回它）	pid / tid（gettid 返回它）
文件描述符	打开的文件对象	描述符表（filedup 复制引用）
调度	—	各自 RUNNABLE，可在不同 CPU 上同时跑